

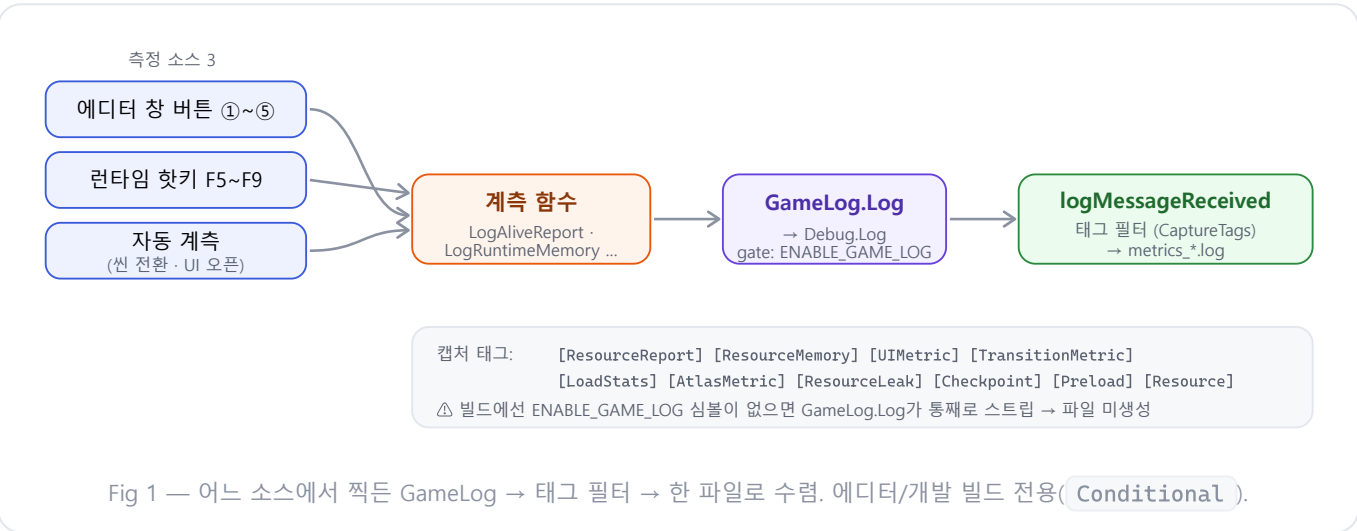
어떻게 측정했나

측정치를 믿기 전에 측정기를 검증하는 루프 — 계측 인프라 · 지표 등급 · 재현 시나리오 · 신뢰도.

이 리팩터의 핵심 가치는 감축량이 아니라 "측정 → 진단 → 구조 → 검증"을 도구의 신뢰도까지 등급 매겨가며 돌린 **규율**이다. 이 문서는 **무엇을 어떻게 잴고, 그 숫자를 얼마나 믿을 수 있는지**를 도식으로 정리한다.

1 계측 파이프라인 — 3 소스 → 자동 파일 기록

콘솔 복사 없이, 태그 붙은 로그만 걸러 파일에 실시간 append.



2 지표를 역할별로 등급화

아무거나 재지 않는다 — 개선을 증명할 것 / 대가 / 회귀 방지 / 참고를 나눈다.

주 지표	메모리 — 버킷별 핸들 메모리(에디터) · 런타임 총량(빌드). 개선을 증명하는 측.
비용	히치(UI 오픈 요청→표시 + 이후 60프레임 최악) · 씬 전환 피크 · 재로드 횟수. 상주를 포기한 대가.
불변	회귀 없음 — 버킷 왕복 정합, 잔존 0, [?] · [ResourceLeak] 0. "딴 거 안 망가뜨렸다"는 증명.
참고	프리로드 시간 — 개선 측이 아니라고 판단 → 참고로 강등(에셋 8~9개라 순차→배치 이득 작음).

3 지표별 측정 방법 (API)

지표	측정 방법	핵심
메모리 (에디터)	<code>EditorUtility.CollectDependencies</code> + <code>GetRuntimeMemorySizeLong</code>	버킷 합계 = Global 우선 귀속(공유 의존성 1회만). <code>SpriteAtlas</code> 는 런타임 페이지 기준 특례
메모리 (빌드)	<code>GetTotalAllocatedMemoryLong</code> + <code>Graphics</code> + <code>Texture.currentTextureMemory</code>	에디터 ×2 CPU 사본 없는 진짜 절대값. 회수 주신호 = 텍스처
히치 (UI 오픈)	<code>realtimeSinceStartup</code> (요청→표시) + <code>UniTask.Yield</code> 로 60프레임 관찰 <code>unscaledDeltaTime</code> 최악	콘텐츠는 표시 이후 프레임에 로드 → 그 히치까지 포착
전환 피크	전환 시작~활성화까지 매 프레임 총할당 샘플링	피크/시작/종료 + 피크 발생 프레임(#N)
로드 통계	캐시 히트 제외, 실제 Addressables 로드만 키별 횟수·시간	재로드 = lazy화 비용의 직접 증거



Fig 2 — 히치는 "요청→표시"만이 아니라 표시 후 60프레임의 최악값까지 분다(콘텐츠는 표시 이후에 로드됨).

4 재현 가능한 시나리오 — 체크포인트 ①~⑤

사람 손 안 타게 고정 시나리오 + 버튼/햅키 마킹. Before/After를 같은 흐름으로.



각 체크포인트 = 구분선 + 버킷 스냅샷 + 메모리 자동 기록 · 팝업이 씬 스크프에 적재됐다 ④에서 회수됨

Fig 3 — 5개 체크포인트가 각각 다른 주장을 검증(기준선/세션비용/캐시히트/회수/flat).

5 단일 변수 A/B — 도구 변화분과 개선분 분리

계측 도구가 중간에 바뀐 구간을 넘어 직접 차감하면 "내 개선"과 "도구 변화"가 섞인다.

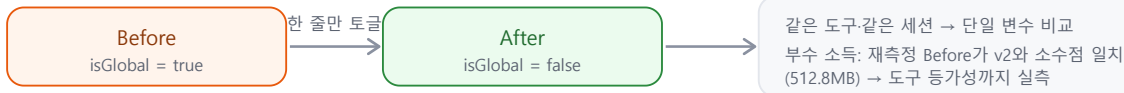


Fig 4 — 배선(isGlobal) 한 줄만 바꿔 같은 도구로 Before를 재측정 = 단일 변수 통제.

6 신뢰도 등급제 — 주장 전에 등급을 매긴다

모든 수치를 같은 무게로 인용하지 않는다.

강

로드 개수·출처 · 피크 발생 프레임 · 재로드 키 목록 · 버킷 라벨 전환 → 그대로 인용

중

히치 ms(n=1, 15ms 이하 차이 주장 금지) · 메모리 MB(에디터 = 증감·비율만, 내부 비교 전용)

약

근거 미비 → 방향만 (예: 원인 미검증 가설)

등급 상황 근거

Addressables Play Mode = **Use Existing Build(실제 번들)** 확인 → Release·언로드·번들 마운트가 실제와 동일 → **증감·비율은 "강"**. 남은 절대값 각주는 **개발 빌드 실측(2026-08-11)**으로 확정.

7 측정기 자체를 검증 (방법론의 정점)

"숫자를 믿기 전에 도구를 의심했다" — 이게 감측 수치보다 희소한 신호.

교차 검증으로 잡은 것 3

- ① **측정 도구 x6.8 과대계상** — 에디트 모드(원본 폴백처럼 보임) vs 플레이 모드(DXT5 페이지 정상) 상반 → 아틀라스가 원본 텍스처까지 세던 계통 오차 발견·보정(436MB → 실제 64MB).
- ② **계측이 죽은 코드와 함께 소멸** — RecordLoad 가 죽은 코드 제거 때 함께 삭제돼 로드 통계가 비어 있던 걸 "왜 이 지표가 비지?"로 역추적.
- ③ **assert 위치 오류** — "이전 썬 미반납"이 아니라 "회전 후 새 적재"를 오탐하던 검사를 await 이전으로 이동.

측정 운영 함정 (같이 기록)

- **TMP 동적 폰트**가 세션마다 절대값을 최대 16MB 흔들(화면에 띄운 글자만 런타임 베이크) → **버킷 합계를 주 지표로**, 폰트 .asset **커밋 금지**(git checkout).
- 에디터는 텍스처 **CPU 사본**까지 계상 → 개발 빌드 실측으로 보정.

한 줄 요약

측정 → 진단 → 구조 → 검증 루프를, **측정기의 신뢰도까지 등급 매겨가며** 돌렸다. 감측(-60%)보다 이 규율이 값이다.